

torch.nn.functional.scaled_dot_product

https://pytorch.org/docs/stable/generated/torch.nn.functional.scaled_dot_product

Description:

`is_causal=False, scale=None, enable_gqa=False) -> Tensor`: Computes scaled dot product attention on query, key and value tensors, using an optional attention mask if passed, and applying dropout if a probability greater than 0.0 is specified. The optional scale argument can only be specified as a keyword argument. This function is beta and subject to change.

Function Signature

```
torch.nn.functional.scaled_dot_product_attention()#  
scaled_dot_product_attention(query, key, value,  
attn_mask=None, dropout_p=0.0,
```

Parameters

Returns

Return type

Returns:

output (Tensor)

Code Examples

```
# Efficient implementation equivalent to the following:
def scaled_dot_product_attention(query, key, value,
    attn_mask=None, dropout_p=0.0,
        is_causal=False, scale=None, enable_gqa=False) ->
torch.Tensor:
    L, S = query.size(-2), key.size(-2)
    scale_factor = 1 / math.sqrt(query.size(-1)) if scale is
None else scale
    attn_bias = torch.zeros(L, S, dtype=query.dtype,
device=query.device)
    if is_causal:
        assert attn_mask is None
        temp_mask = torch.ones(L, S,
dtype=torch.bool).tril(diagonal=0)
        attn_bias.masked_fill_(temp_mask.logical_not(), float("-
inf"))

    if attn_mask is not None:
        if attn_mask.dtype == torch.bool:
            attn_bias.masked_fill_(attn_mask.logical_not(),
float("-inf"))
        else:
            attn_bias = attn_mask + attn_bias

    if enable_gqa:
        key =
key.repeat_interleave(query.size(-3)//key.size(-3), -3)
```

```

        value =
value.repeat_interleave(query.size(-3)//value.size(-3), -3)

    attn_weight = query @ key.transpose(-2, -1) * scale_factor
    attn_weight += attn_bias
    attn_weight = torch.softmax(attn_weight, dim=-1)
    attn_weight = torch.dropout(attn_weight, dropout_p,
train=True)
    return attn_weight @ value

```

```

class MyModel(nn.Module):
    def __init__(self, p=0.5):
        super().__init__()
        self.p = p

    def forward(self, ...):
        return F.scaled_dot_product_attention(...,
            dropout_p=(self.p if self.training else 0.0))

```

```

>>> # Optionally use the context manager to ensure one of the
fused kernels is run
>>> query = torch.rand(32, 8, 128, 64, dtype=torch.float16,
device="cuda")
>>> key = torch.rand(32, 8, 128, 64, dtype=torch.float16,
device="cuda")
>>> value = torch.rand(32, 8, 128, 64, dtype=torch.float16,
device="cuda")
>>> with sdpa_kernel(backends=[SDPBackend.FLASH_ATTENTION]):
>>>     F.scaled_dot_product_attention(query, key, value)

```

```
>>> # Sample for GQA for llama3
>>> query = torch.rand(32, 32, 128, 64, dtype=torch.float16,
device="cuda")
>>> key = torch.rand(32, 8, 128, 64, dtype=torch.float16,
device="cuda")
>>> value = torch.rand(32, 8, 128, 64, dtype=torch.float16,
device="cuda")
>>> with sdpa_kernel(backends=[SDPBackend.MATH]):
>>>
F.scaled_dot_product_attention(query, key, value, enable_gqa=True)
```

Notes & Warnings

 **WARNING** Warning This function is beta and subject to change.

 **WARNING** This function always applies dropout according to the specified dropout_p argument. To disable dropout during evaluation, be sure to pass a value of 0.0 when the module that makes the function call is not in training mode. For example: class MyModel(nn.Module): def __init__(self, p=0.5): super().__init__() self.p = p def forward(self, ...): return F.scaled_dot_product_attention(..., dropout_p=(self.p if self.training else 0.0))

NOTE There are currently three supported implementations of scaled dot product attention: FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning Memory-Efficient Attention A PyTorch implementation defined in C++ matching the above formulation The function may call optimized kernels for improved performance when using the CUDA backend. For all other backends, the PyTorch implementation will be used. All implementations are enabled by default. Scaled dot product attention attempts to automatically select the most optimal implementation based on the inputs. In order to provide more fine-grained control over what implementation is used, the following functions are provided for enabling and disabling implementations. The context manager is the preferred mechanism:

NOTE In some circumstances when given tensors on a CUDA device and using CuDNN, this operator may select a nondeterministic algorithm to increase performance. If this is undesirable, you can try to make the operation deterministic (potentially at a performance cost) by setting `torch.backends.cudnn.deterministic = True`. See Reproducibility for more information.

Detailed Documentation

Documentation

`is_causal=False, scale=None, enable_gqa=False`) -> Tensor:

Computes scaled dot product attention on query, key and value tensors, using an optional attention mask if passed, and applying dropout if a probability greater than 0.0 is specified. The optional scale argument can only be specified as a keyword argument.

This function is beta and subject to change.

This function always applies dropout according to the specified dropout_p argument. To disable dropout during evaluation, be sure to pass a value of 0.0 when the module that makes the function call is not in training mode.

There are currently three supported implementations of scaled dot product attention:

FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning

Memory-Efficient Attention

A PyTorch implementation defined in C++ matching the above formulation

The function may call optimized kernels for improved performance when using the CUDA backend. For all other backends, the PyTorch implementation will be used.

All implementations are enabled by default. Scaled dot product attention attempts to automatically select the most optimal implementation based on the inputs. In order to provide more fine-grained control over what implementation is used, the following functions are provided for enabling and disabling implementations. The context manager is the preferred mechanism:

`torch.nn.attention.sdpa_kernel()`: A context manager used to enable or disable any of the implementations.

`torch.backends.cuda.enable_flash_sdp()`: Globally enables or disables FlashAttention.

`torch.backends.cuda.enable_mem_efficient_sdp()`: Globally enables or disables

Memory-Efficient Attention.

`torch.backends.cuda.enable_math_sdp()`: Globally enables or disables the PyTorch C++ implementation.

Each of the fused kernels has specific input limitations. If the user requires the use of a specific fused implementation, disable the PyTorch C++ implementation using `torch.nn.attention.sdpa_kernel()`. In the event that a fused implementation is not available, a warning will be raised with the reasons why the fused implementation cannot run.

Due to the nature of fusing floating point operations, the output of this function may be different depending on what backend kernel is chosen. The c++ implementation supports `torch.float64` and can be used when higher precision is required. For math backend, all intermediates are kept in `torch.float` if inputs are in `torch.half` or `torch.bfloat16`.

For more information please see Numerical accuracy

Grouped Query Attention (GQA) is an experimental feature. It currently works only for Flash_attention and math kernel on CUDA tensor, and does not support Nested tensor. Constraints for GQA:

`number_of_heads_query % number_of_heads_key_value == 0` and,

`number_of_heads_key == number_of_heads_value`

In some circumstances when given tensors on a CUDA device and using CuDNN, this operator may select a nondeterministic algorithm to increase performance. If this is undesirable, you can try to make the operation deterministic (potentially at a performance cost) by setting `torch.backends.cudnn.deterministic = True`. See Reproducibility for more information.

query (Tensor) – Query tensor; shape $(N, \dots, H_q, L, E)$

key (Tensor) – Key tensor; shape $(N, \dots, H, S, E)$

value (Tensor) – Value tensor; shape $(N, \dots, H, S, E_v)$

attn_mask (optional Tensor) – Attention mask; shape must be broadcastable to the shape of attention weights, which is $(N, \dots, L, S)$. Two types of masks are supported. A boolean mask where a value of True indicates that the element should take part in attention. A float mask of the same type as query, key, value that is added to the attention score.

dropout_p (float) – Dropout probability; if greater than 0.0, dropout is applied

is_causal (bool) – If set to true, the attention masking is a lower triangular matrix when the mask is a square matrix. The attention masking has the form of the upper left causal bias due to the alignment (see `torch.nn.attention.bias.CausalBias`) when the mask is a non-square matrix. An error is thrown if both `attn_mask` and `is_causal` are set.

scale (optional python:float, keyword-only) – Scaling factor applied prior to softmax. If None, the default value is set to $\frac{1}{\sqrt{E}}$.

enable_gqa (bool) – If set to True, Grouped Query Attention (GQA) is enabled, by default it is set to False.

Attention output; shape $(N, \dots, H_q, L, E_v)$

N:Batch size...:Any number of other batch dimensions (optional)
N: \text{Batch size} ... :
\text{Any number of other batch dimensions}
(optional)}N:Batch size...:Any number of other batch dimensions (optional)

S:Source sequence length
S: \text{Source sequence length}S:Source sequence length

L:Target sequence lengthL: \text{Target sequence length}L:Target sequence length

E:Embedding dimension of the query and keyE: \text{Embedding dimension of the query and key}E:Embedding dimension of the query and key

Ev:Embedding dimension of the valueEv: \text{Embedding dimension of the value}Ev:Embedding dimension of the value

Hq:Number of heads of queryHq: \text{Number of heads of query}Hq:Number of heads of query

H:Number of heads of key and valueH: \text{Number of heads of key and value}H:Number of heads of key and value